

Dynamic Programming

Memoization

Content

Two Parts

Part 1: Memoization

Part 2: Dynamic Programming

Content

- Two Parts

Part 1: Memoization

Fibonacci

Combinations nCr (HW)

Part 2: Dynamic Programming

Weighted Independent Set Problem

Knapsack Problem

Sequence Alignment (Genome) – Reading

Knapsack

Given a set of items, each with a weight and a value, determine the which items to include in a collection so that the total weight is less than or equal to a given limit and the total value is as large as possible.

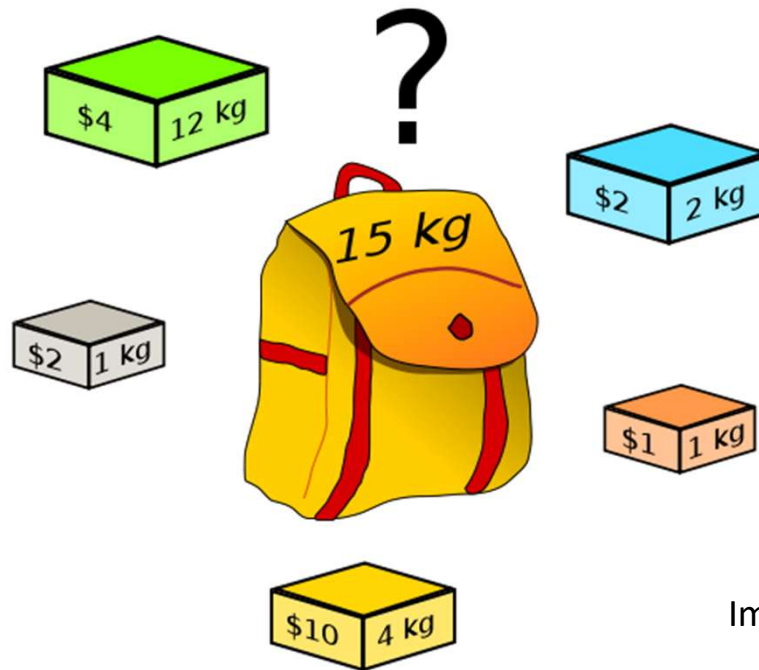


Image Source: Wikipedia

Sequence Alignment for genomic data

- Find similarities in the amino acid sequences of two proteins

Histone H1 (residues 120-180)	
HUMAN	KKASKPKKAASKAPTKKPKATPVKKAKKKLAATPKKAKKPKTVKAKPVKASKPKKAKPVK
CHIMP	KKASKPKKAASKAPTKKPKATPVKKAKKKLAATPKKAKKPKTVKAKPVKASKPKKAKPVK
MOUSE	KKAAKPKKAASKAPSKKPKATPVKKAKKKPAATPKKAKKPKVVKVPVKASKPKKAKTVK
RAT	KKAAKPKKAASKAPSKKPKATPVKKAKKKPAATPKKAKKPKIVKVPVKASKPKKAKPVK
COW	KKAAKPKKAASKAPSKKPKATPVKKAKKKPAATPKKTKKPKTVKAKPVKASKPKKTKPVK
	:**:***** *****:***** **.******:*
NON-CONSERVED AMINO ACIDS	Conservative Conservative Non-conservative Conservative Non-conservative Semi-conservative Non-conservative

Image Source: Wikipedia

Memoization

- Keeping a memo

How to use memoization to speedup Fibonacci method

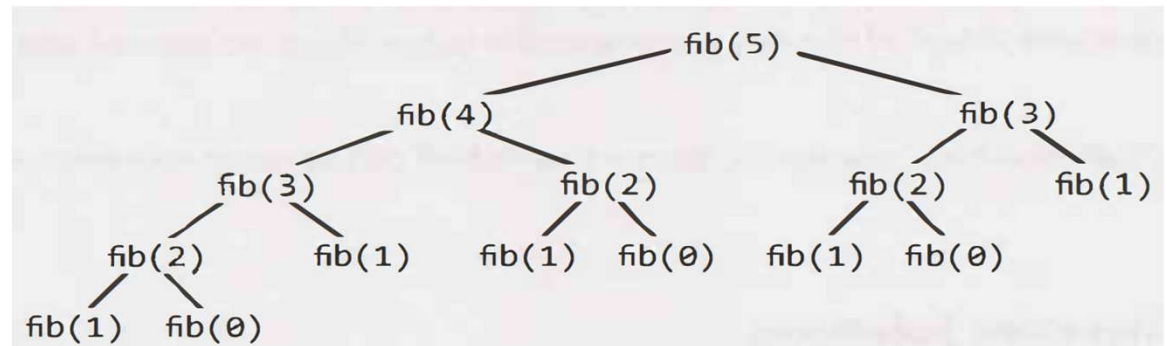
- Inefficient algorithms => Efficient algorithms

Compute the ith Fibonacci number

```
int fibonacci( int i)
{
    if (i == 0)
        return 0;
    if (i == 1)
        return 1;
    return  fibonacci( i-1 ) +  fibonacci( i-2 );
}
```

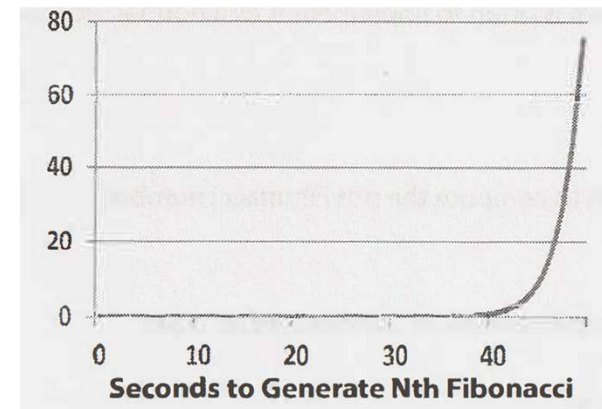
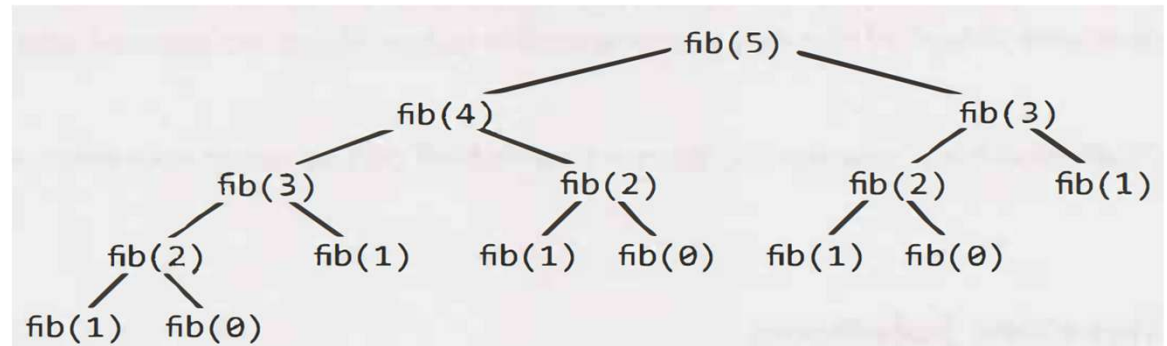
Compute the ith Fibonacci number

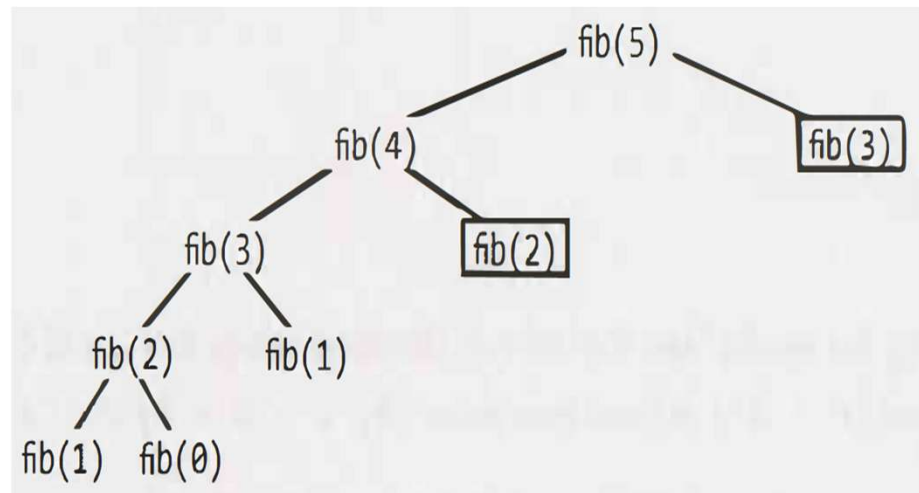
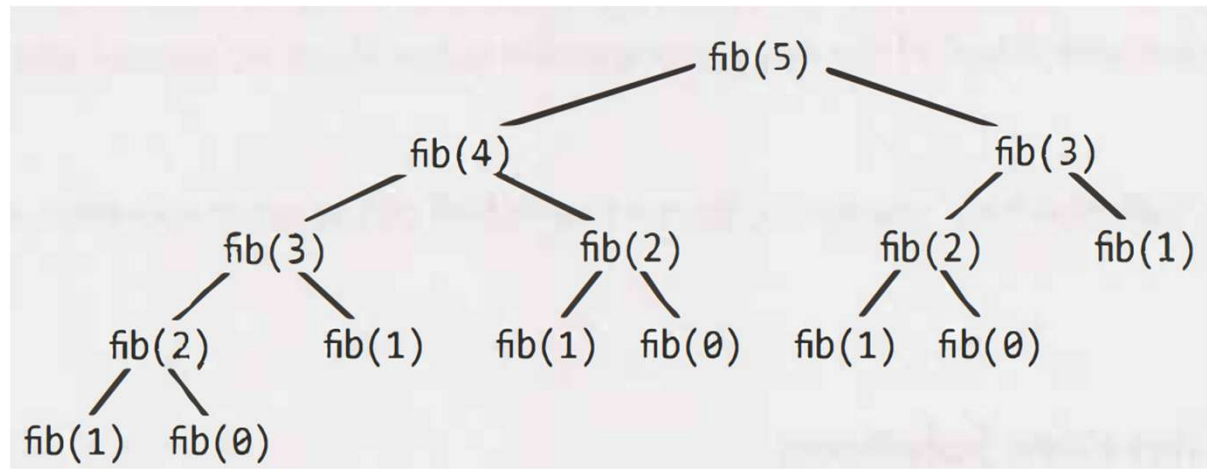
```
int fib( int i )  
{  
    if (i == 0)  
        return 0;  
    if (i == 1)  
        return 1;  
    return fib ( i-1 ) +  fib( i-2 );  
}
```



Compute the ith Fibonacci number

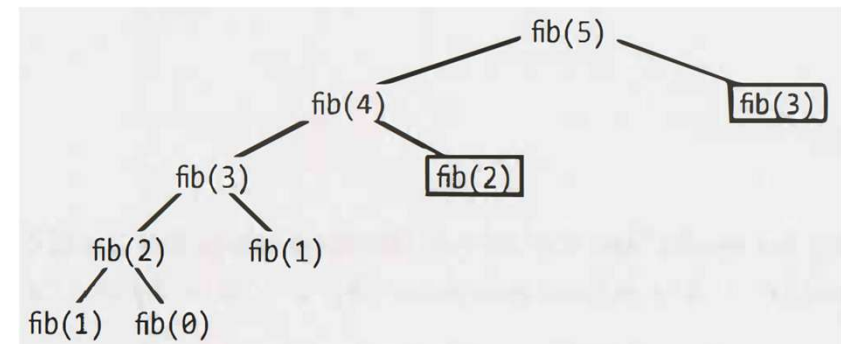
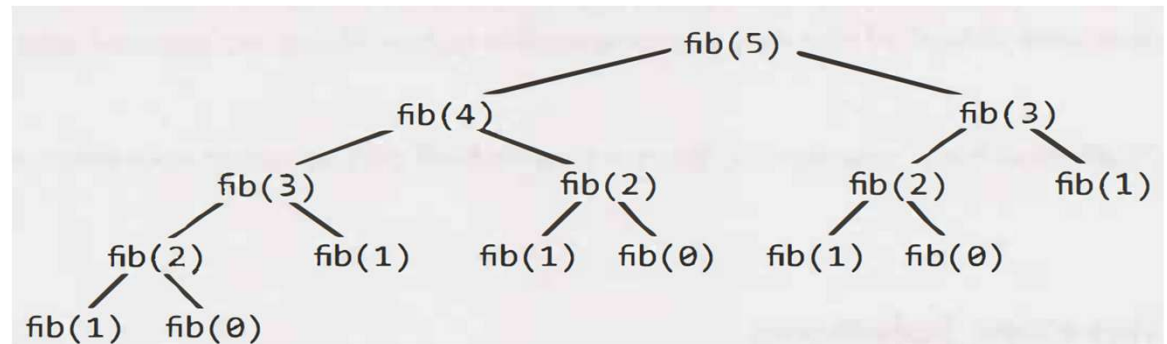
```
int fib( int i )  
{  
    if (i == 0)  
        return 0;  
    if (i == 1)  
        return 1;  
    return fib ( i-1 ) +  fib( i-2 );  
}
```





Compute the ith Fibonacci number

```
int fib( int i )  
{  
    if (i == 0)  
        return 0;  
    if (i == 1)  
        return 1;  
    return fib( i-1 ) + fib( i-2 );  
}
```



Memoized version of Fibonacci

```
int fib( int i)
{
    int []cache = new int[ i+1 ];

    return fib( i, cache);
}
```

```
int fib( int i, int []cache )
{
    if (i == 0)  return 0;

    if (i == 1)  return 1;

    if(cache[i] == 0) {
        cache[i] =  fib( i-1, cache )  +  fib( i-2, cache );
    }
    return cache[i];
}
```

index	0	1	2	3	4	5
cache	0	0	0	0	0	0

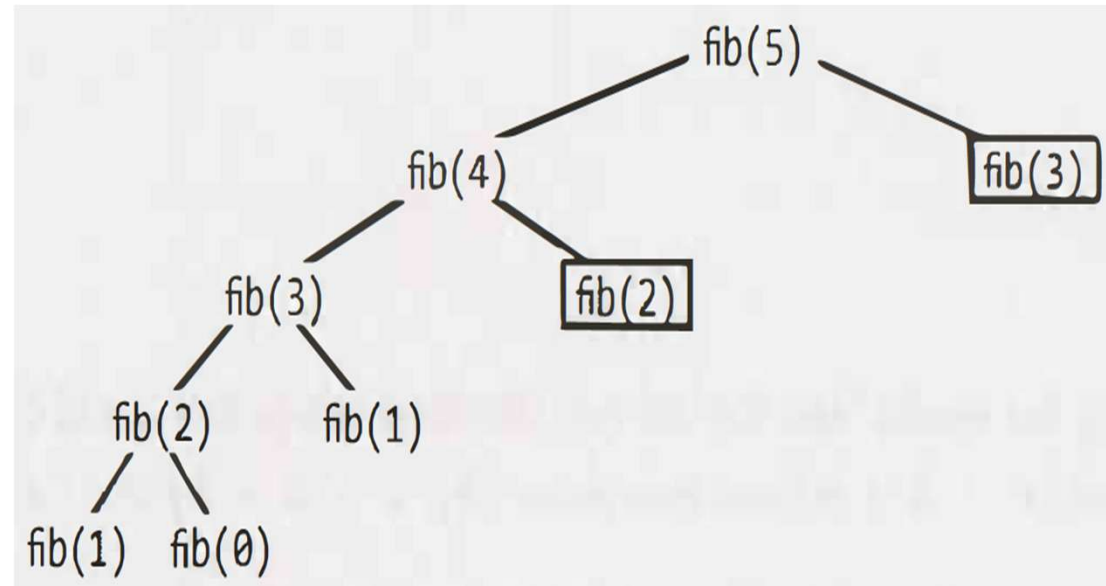
```

int fib( int i, int []cache )
{
    if (i == 0)    return 0;

    if (i == 1)    return 1;

    if(cache[i] == 0)
    {
        cache[i] =  fib( i-1, cache ) +  fib( i-2, cache );
    }
    return cache[i];
}

```



Compute the ith Fibonacci number

```
int fibonacci( int i )
{
    if (i == 0) return 0;

    if (i == 1) return 1;

    return fib(i-1) + fib(i-2);
}
```

Memoization

```
int fib( int i )
{
    int []cache = new int[i+1];
    return fib(n, cache);
}

int fib( int i, int []cache )
{
    if (i == 0) return 0;
    if (i == 1) return 1;

    if(cache[i] == 0)
    {
        cache[i] = fib( i-1, cache ) + fib( i-2, cache );
    }
    return cache[i];
}
```

Combinations

- Group consist of three members A, B, C
- Combinations of size 3
 - 1 combination: all members included $\Rightarrow (A, B, C)$

Combinations

- Group consist of three members A, B, C
- Combinations of size 3
 - 1 combination: all members included $\Rightarrow (A, B, C)$
- Combinations of size 2
 - 3 combinations: (A, B) , (B,C) , (A,C)

Combinations

- Group consist of three members A, B, C
- Combinations of size 3
1 combination: all members included => (A, B, C)
- Combinations of size 2
3 combinations: (A, B) , (B,C), (A,C)
- **Combinations of size 1**
3 combinations A,
B,
C

Combinations: from n choose r

- How many combinations of a certain size can be made out of a group of items?
- From 20 students, form a panel consisting of five students. How many such panels are possible?
- C is the total number of combinations
- group is the total size of the group to pick from,
- members is the size of each subgroup (panel)
- group $>$ members

Recursive example (3, 2)

A, B, C $\Rightarrow$ (A, B) , (B,C), (A,C)

A is in 2 subgroups and not in one of them

- Combinations (3, 2)

= Combinations (2, 1) + Combinations (2, 2)

= 2 + 1

= 3

How many combinations, C?

C is the total number of combinations

group is the total size of the group to pick from,

members is the size of each subgroup (panel)

= group, if members = 1

C (groups, members) = 1, if members = group

= C (group – 1, members – 1) + C (group-1, members)
if group > members > 1

Combinations method

```
int combinations( int group, int members )
{
    if (members == 1)
        return group;                // Base case 1
    else if (members == group)
        return 1;                    // Base case 2
    else
        return combinations(group - 1, members - 1) + combinations(group - 1, members);
}
```

Combinations (3, 2)

/

\

Combinations (2, 1)

Combinations (2, 2)

Combinations (3, 2)

/

\

Combinations (2, 1)

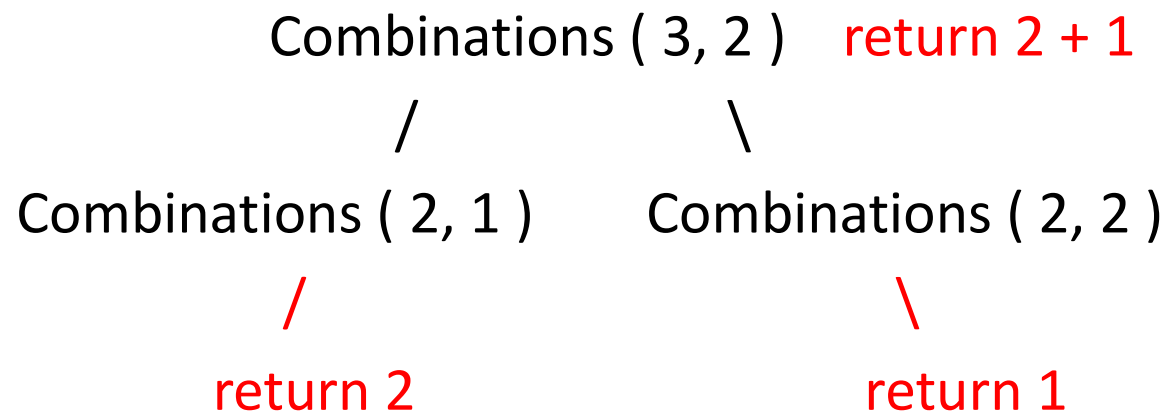
Combinations (2, 2)

/

return 2

\

return 1



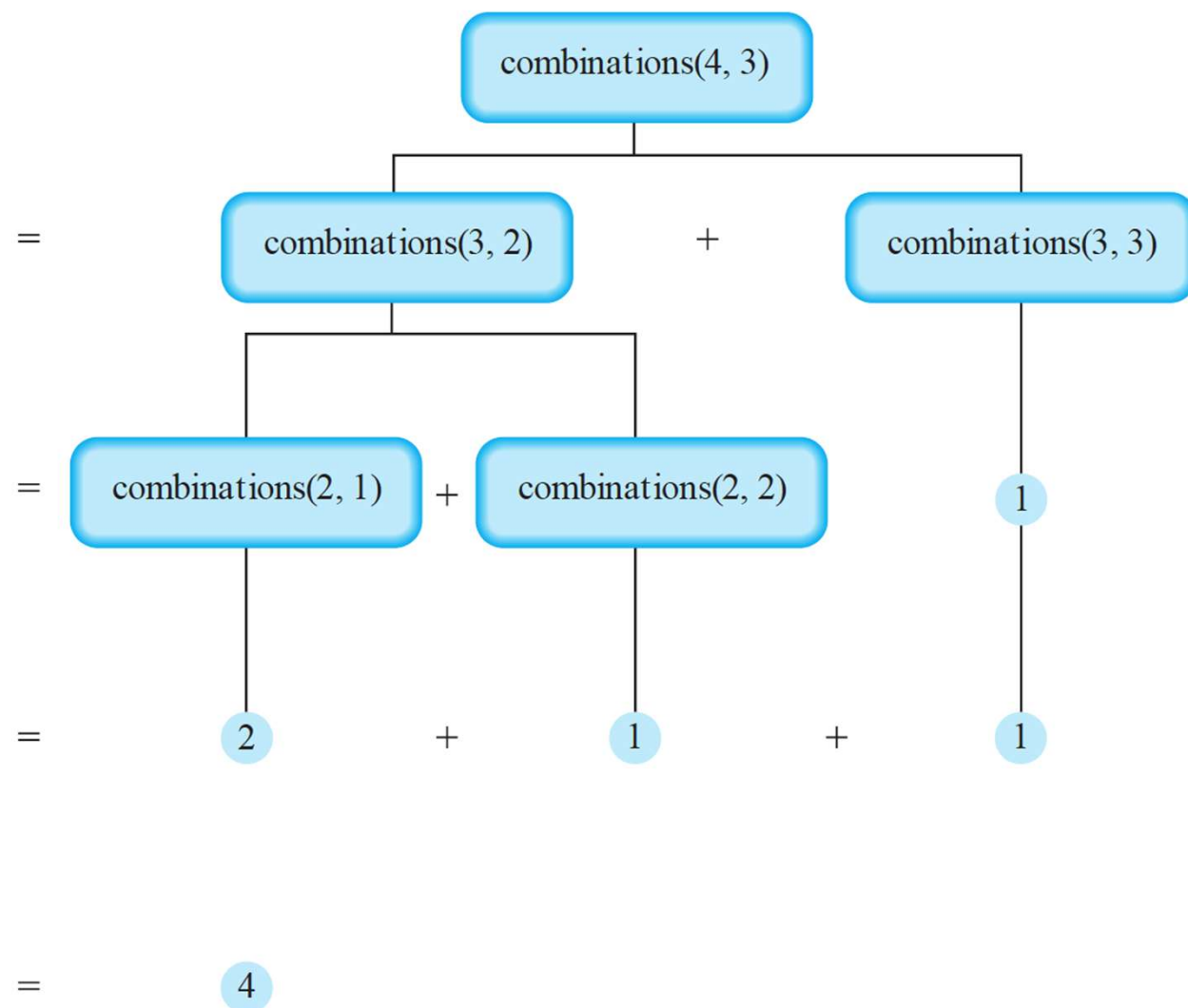


Figure 4.2 Calculating $\text{combinations}(4, 3)$

Combinations HW

- Use extra space to memoize (like Fibonacci optimization)
- Make algorithm run faster – by how much?